AI ENGINEERING
AI-Powered products

MÓDULO 3: Data-driven AI

# Sesión 7: Embeddings y representación vectorial

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

# Quién os acompaña hoy

Sesión 7 · Módulo 3 · Data-driven AI

## Julián Fernández

Informático · fundador de Búho AI

- Transformo operativas complejas en procesos eficientes con IA y software a medida.
- Profesor de IA en LIDR, EBF y UCAM HiTech.
- Enfoque **transparente y ético**: medir antes de opinar, y si un dato no se sabe, no se inventa.

linkedin.com/in/julianfernandez-iaenmurcia

## Teaching Assistant

George Kutsia Andguladze

george@lidr.co · dudas técnicas en el canal asíncrono

### Cómo es la clase

- Flipped: los artículos y el ejercicio previo ya están hechos.
- **No se teclea código**: se lee el que ya existe y se ejecuta.
- Hoy, mucha terminal: ocho estrategias de chunking medidas en directo.

"Cambiar el chunking mueve la calidad de un RAG tanto como cambiar de modelo. **Hoy lo medimos sobre nuestros propios datos.**"

# Lo que vamos a ver hoy

Del texto al vector: qué es un embedding y, sobre todo, cómo partir los documentos

**Apertura** · Dónde estamos en el pipeline RAG y con qué corpus trabajamos

**Dudas del ejercicio** · Vuestros números del sanity check y cinco errores reales

**Embeddings** · Qué son, cómo se miden, qué modelo elegir y Matryoshka

**Chunking I · mecánicas** · Fixed-size, recursive, sentence-window y structural

**Chunking II · semánticas** · Semantic, propositional y Contextual Retrieval

**Chunking III · avanzadas** · Hierarchical (parent-child) y late chunking

**Cierre** · El cuadro de mando para elegir · puente a la Sesión 8

**Al salir de hoy sabréis…**

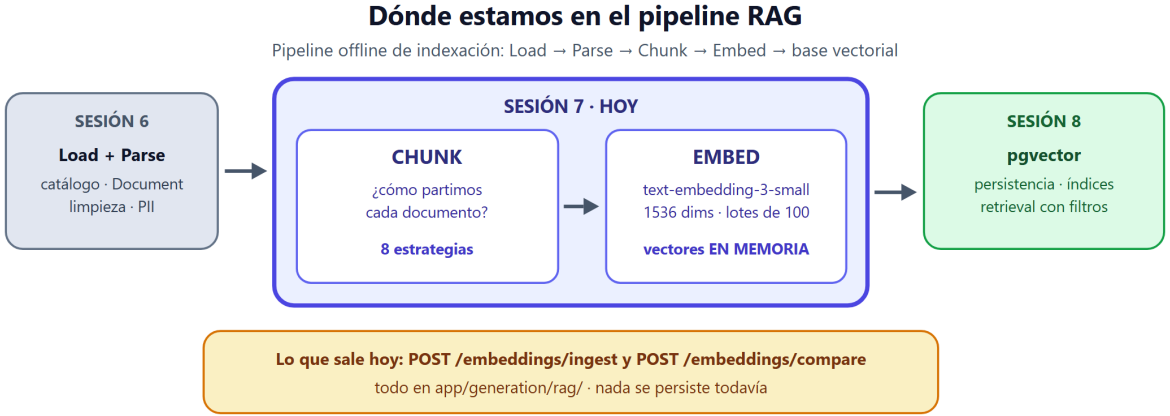
- Explicar qué mide un coseno y qué no.
- Elegir modelo de embeddings con criterios.
- Comparar 8 estrategias de chunking con datos.
- Defender una elección ante coste y calidad.

## Cómo trabajamos

Cada estrategia: **la idea → dónde vive en el código → demo → la pregunta que tenéis que saber responder**.

# Hoy: Chunk + Embed

La mitad central del pipeline offline de indexación




## pgvector, en la Sesión 8

Hoy los vectores viven **en memoria** y se devuelven por HTTP. Así liberamos el tiempo para el chunking, que es donde de verdad se mueve la calidad de un RAG.

## El shape manda

El corpus de hoy está organizado por **componentes**, no por las fases de la S6. La forma de tus datos condiciona la estrategia de chunking que eliges.

Código nuevo de hoy: `app/generation/rag/` · `chunking/` · `embedding/` · `analysis/`

# El corpus de la sesión

data/budgets_sample.json · 17 presupuestos · 60 componentes

**Un presupuesto**

```
{
  "budget_id": "BUD-2024-001",
  "client_metadata": {"name": "FintechCorp",
                      "sector": "finance",
                      "country": "ES"},
  "project_summary": "Mobile banking API with
       OAuth 2.0 authentication and PSD2 compliance",
  "main_technology": "ruby_on_rails",
  "year": 2024,
  "components": [
    {"component_id": "AUTH-001",
     "name": "OAuth 2.0 authentication backend",
     "description": "Implementation of OAuth 2.0
        flows ... JWT-based session management ...",
     "tech_stack": ["ruby_on_rails", "postgresql",
                    "redis"],
     "complexity": "high",
     "estimated_hours": 120},
    ...  PSD2-002 · TXN-003 · NOTIF-004
  ]
}
```

## components[ ] es la unidad de trabajo

Todo lo que hacemos hoy gira sobre ese array: cada componente es una pieza de software con descripción, stack, complejidad y horas estimadas.

## Dos presupuestos con trampa, a propósito

- **BUD-2024-016**: un componente con una descripción gigante (`MONO-001`, 1391 tokens).
- **BUD-2024-017**: descripciones de una sola línea.

## Para pensar

Si el schema de los datos no coincide con lo prometido, alguien escribe el adaptador. **¿Quién lo presupuesta?**

L1DR

# Tres arquitecturas, una base común

Por qué reorganizamos el proyecto en esta sesión · de carpetas planas a arquitectura por capas

ESTRUCTURA DEL PROYECTO

```
app/
├── foundation/        plomería sin opinión
├── domain/            schemas + el "conductor"
├── generation/        las 3 arquitecturas que COMPONEN
│   ├── cag/           CAG   caches (ya construido)
│   ├── rag/           RAG   ● LO DE HOY
│   │   ├── chunking/ · embedding/ · analysis/
│   │   ├── store/         ← slot reservado S8
│   │   └── retriever.py   ← slot reservado S8
│   ├── agentic/       AGENTIC   boss + critic (futuro)
│   └── conversation/
├── ingestion/         pipeline batch que alimenta RAG
└── api/               transporte (routers finos)
```

REGLA: Los hermanos de generation/ no se importan entre sí · solo se encuentran en el conductor

## Qué cambia exactamente

El estimador apila tres arquitecturas de IA de forma aditiva a lo largo del máster — CAG, RAG (hoy) y agentic (futuro).

A partir de hoy esa historia se ve en la estructura.

## Por qué ahora

Las carpetas planas escondían el patrón: caches partidas, RAG aislado, lo agéntico mezclado en services.

Antes de añadir más piezas, ponemos cada cosa en su capa.

## Lo de hoy en este mapa

Todo lo que tocamos en la S7 es exactamente el contenido de **`app/generation/rag/`**

store/ y retriever.py se rellenan en la S8.

Frase para defender ante un stakeholder

"No reescribimos lógica: movimos las piezas a su capa. La arquitectura ahora se lee de un vistazo y cada cosa nueva sabe dónde va."

# El ejercicio previo: vuestro sanity check

text-embedding-3-small · coseno calculado a mano · tres parejas

| Pareja | Texto 1 | Texto 2 | Coseno | Esperado |
|---|---|---|---|---|
| **A · cercanas** | OAuth 2.0 authentication backend with JWT tokens for fintech mobile app | Authorization service using JSON Web Tokens for a banking application | **0,596** | > 0,6 |
| **B · sin relación** | (la misma) | Database migration from MySQL to PostgreSQL with zero downtime | **0,192** | < 0,4 |
| **C · genéricas** | Backend services | API development | **0,541** | — |

### A: 0,596 < 0,6

Mismo concepto con **cero palabras en común**. El modelo casi lo salva. Los umbrales dependen del modelo y de la redacción: no son constantes universales.

### B: 0,19

Lo esperado: temas sin relación quedan claramente separados.

Ojo: 0,2–0,5 entre textos sin relación es normal en estos modelos.

### C: 0,54, el aviso

Dos frases cortas y vagas puntúan casi como A. **El texto corto y genérico no discrimina.** Por eso cada chunk lleva una cabecera con proyecto, sector y tecnología.

L1DR

# Cinco patrones de error reales

Ordenados por impacto en producción

1

## Sin validación Pydantic

Síntoma: `KeyError` opaco en producción cuando falta un campo

Resuelto en · app/generation/rag/schemas.py

2

## Gestión naïve de tokens

Síntoma: chunk excede contexto del modelo · `422` de la API de embeddings

Resuelto en · app/generation/rag/chunking/base.py :: count_tokens

3

## Sin batching

Síntoma: un request por chunk · latencia que se dispara · throughput inviable

Resuelto en · embedder.py :: BATCH_SIZE = 100 + embed_many

4

## Sin backoff exponencial

Síntoma: caída ante el primer `RateLimitError` · sin resiliencia

Resuelto en · embedder.py :: _RETRY_BACKOFF_SECONDS = (1, 2, 4)

5

## Dimensionalidad inconsistente

Síntoma: mezclar vectores de 1536 y 768 en el mismo índice rompe la similitud

Resuelto en · schemas.py :: EmbeddedChunk con campo embedding tipado

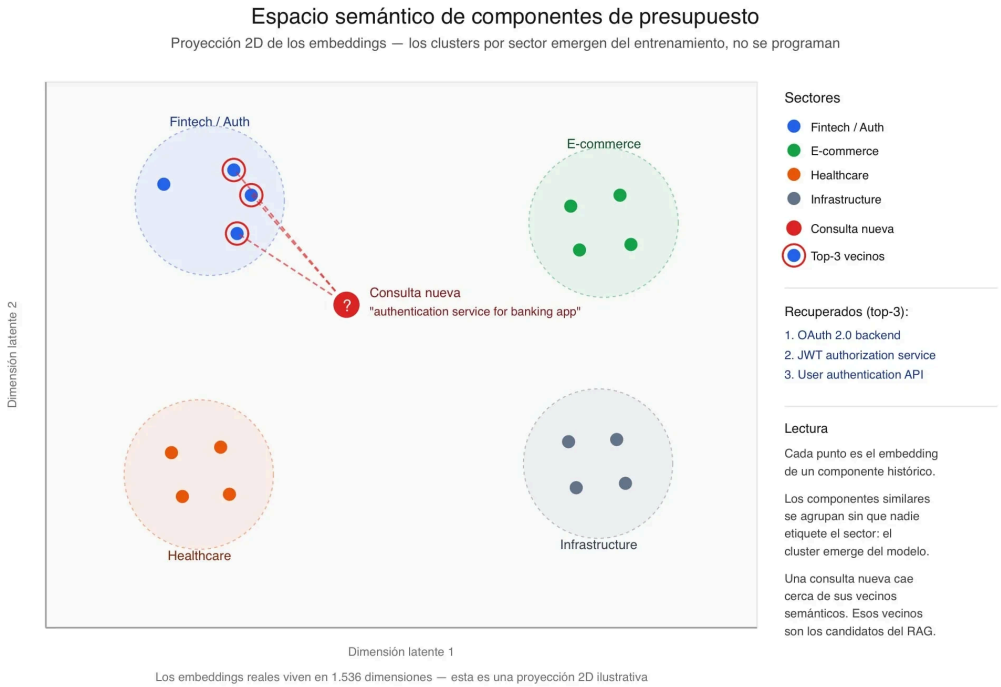
### Pregunta gancho del bloque

"Vuestro chunker no truncó
esa descripción de 1.391 tokens.
¿Está bien o está mal?"

Respuesta esperada: depende.
Hoy no rompe. Mañana sí.

# ¿Qué es un embedding?

Una función texto → vector de dimensión fija: textos parecidos, vectores cercanos




## La propiedad que importa

"authentication service" encuentra "OAuth 2.0 backend" o "JWT authorization module" **aunque no compartan palabras**. Eso es búsqueda semántica.

## Aprendizaje contrastivo

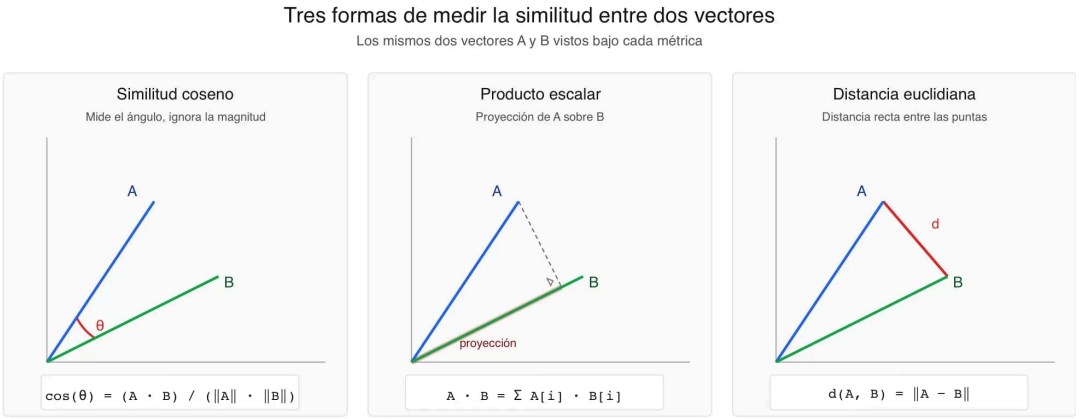
Millones de tripletes **ancla** / **positivo** / **negativo**. Lo que se etiqueta como "positivo" define qué entiende el modelo por similar.

## Las dimensiones emergen

Ninguna dimensión suelta significa nada legible: lo que tiene sentido son **direcciones y distancias relativas**.

# Tres formas de medir la similitud

Con vectores normalizados, las tres ordenan igual




Para vectores normalizados (lo habitual en modelos modernos), las tres métricas ordenan los resultados de la misma forma

| Métrica | Fórmula | Rango | Rasgo |
|---|---|---|---|
| **Coseno** | `A·B / (‖A‖ ‖B‖)` | –1 a 1 | mide el ángulo: un documento largo no sale penalizado frente a una consulta corta |
| **Producto escalar** | `Σ A[i]·B[i]` | sin acotar | el más barato (no calcula normas) |
| **Euclídea** | `√Σ (A[i]−B[i])²` | 0 a ∞ | distancia recta entre las puntas |

**text-embedding-3-small devuelve vectores de norma 1** → producto escalar = coseno, y `euclídea² = 2 − 2·coseno`. La métrica se elige por la *model card*, no es una decisión de arquitectura.

# Lo que un embedding NO resuelve

Saberlo de antemano evita diseñar el sistema equivocado

## 1 · Números, fechas e identificadores

"Presupuestos de 2024" o "más de 100 horas" no se resuelven por similitud: se resuelven con **filtros sobre el metadata** (Sesión 8).

## 2 · Palabras raras exactas

Nombres propios, códigos, siglas: ahí **BM25** gana. Por eso existe la búsqueda híbrida (Sesión 10).

## 3 · La concentración de distancias

En alta dimensión, 0,2–0,5 entre textos sin relación es normal. **Calibra los umbrales con tu dataset**: "> 0,7" no significa nada por sí solo.

## 4 · El modelo pesa más que la métrica

Pasar de coseno a producto escalar no cambia nada. Pasar de un modelo solo-inglés a uno multilingüe, sí.

# Elegir modelo de embeddings

Precios y scores cambian cada pocos meses: verificar antes de decidir

| Modelo | Dims | $ / M tokens | Notas |
|---|---|---|---|
| **text-embedding-3-small** | 1536 | 0,02 | el caballo de batalla · Matryoshka |
| text-embedding-3-large | 3072 | 0,13 | mejor calidad, 6,5× más caro |
| Cohere embed-v3 | — | 0,10 | fuerte en multilingüe |
| Voyage voyage-3-large | — | ≈ Cohere | optimizado para retrieval |
| BAAI/bge-m3 | 1024 | 0 (GPU) | open source, multilingüe |
| all-MiniLM-L6-v2 | 384 | 0 (CPU) | inglés, ideal para prototipos |

## Cinco criterios

- **Dimensiones**: con cientos de millones de vectores, 1536 vs 768 decide si cabe en RAM.
- **Idioma**: corpus en inglés, briefs en español → multilingüe.
- **Dominio**: no hay modelo de "presupuestos": se valida sobre tus datos.
- **Hosting y coste**: API vs self-hosted.
- **Licencia y privacidad**: banca, sanidad, defensa → self-hosted.

## Nuestra decisión

`text-embedding-3-small`, 1536 dims: calidad sobrada, coste despreciable.

L1DR

# Matryoshka: la palanca de dimensiones

Mismo modelo · misma señal · la mitad de storage

| Modelo | Dimensiones | Latencia media | Cosine(c0, c1) | Δ Calidad |
|---|---|---|---|---|
| **`text-embedding-3-small`** @ 1536d nativo | 1536 | 879.8 ms | 0.9119 | baseline |
| **`text-embedding-3-small`** @ 768d (Matryoshka) | 768 | 699.9 ms | 0.9117 | –0.0002 |

### ½ storage

Mitad de dimensiones = mitad de bytes por vector en la BBDD

Para 1M de chunks: GB que entran en RAM o que no entran

### Sin pérdida real

Cosine entre dos componentes similares: idéntica hasta el cuarto decimal

0.9119 vs 0.9117 — ruido

### Un parámetro

Truncar dimensiones no es un modelo nuevo. Es el argumento **`dimensions=768`**

Palanca futura para S8

LOCKED PARA EL PROYECTO

text-embedding-3-small @ 1536d · API ya configurada · coste despreciable · Matryoshka como palanca futura

# ¿Por qué no se comparan vectores de distinta dimensión?

Pregunta frecuente: 1536 contra 768 no da un número malo, da un número sin sentido

## La matemática

El producto escalar `A·B = Σ A[i]·B[i]` empareja la coordenada *i* de uno con la *i* del otro. Con 1536 frente a 768, **las 768 últimas no tienen pareja**: la operación no está definida. Son espacios distintos, como sumar metros con euros.

**Si tienes suerte: el error salta**

```
>>> np.dot(v1536, v768)
ValueError: shapes (1536,) and (768,) not aligned

-- pgvector (Sesión 8)
ERROR: different vector dimensions 1536 and 768
ERROR: expected 1536 dimensions, not 768
```

**Si no la tienes: nuestro coseno en Python puro**

```
# app/generation/rag/analysis/similarity.py
dot = sum(x * y for x, y in zip(a, b))
#                           ^^^ zip se para en la
#                     lista más corta: sin error
norm_a = math.sqrt(sum(x * x for x in a))   # 1536
norm_b = math.sqrt(sum(y * y for y in b))   #  768
```

## El bug silencioso, medido

Un vector de 1536 contra **sus propias** 768 primeras dimensiones:

**0,689** en vez de **1,0**

El mismo texto "se parece" un 69 % a sí mismo. Nada falla, y el ranking queda corrompido.

**Cómo se evita:** un solo modelo y una sola dimensión por índice, guardados junto al vector · `assert len(a) == len(b)` · con Matryoshka, truncar **los dos** lados y renormalizar · y ojo: dos modelos distintos con 1536 dims tampoco son comparables.

# Chunk, token y vector

Tres palabras que vamos a usar todo el rato

**Documento** un presupuesto entero → **Chunks** los trozos en que lo partimos → **Tokens** cómo mide el modelo cada trozo → **Vectores** 1536 números por chunk

## Chunk

El **trozo de documento** que se indexa y se recupera como una unidad. Es lo que el RAG le entrega al LLM como contexto.

En nuestro proyecto, por defecto, un componente del presupuesto.

## Token

La unidad en la que el modelo cuenta el texto: **unos ¾ de palabra** en inglés (tokenizador `cl100k_base`, con tiktoken).

Medimos los chunks en tokens, no en caracteres: es lo que cobra la API y lo que limita el modelo (8191 tokens de entrada como máximo).

## Vector (embedding)

Los **1536 números** que el modelo devuelve para un chunk. **Un chunk → un vector.**

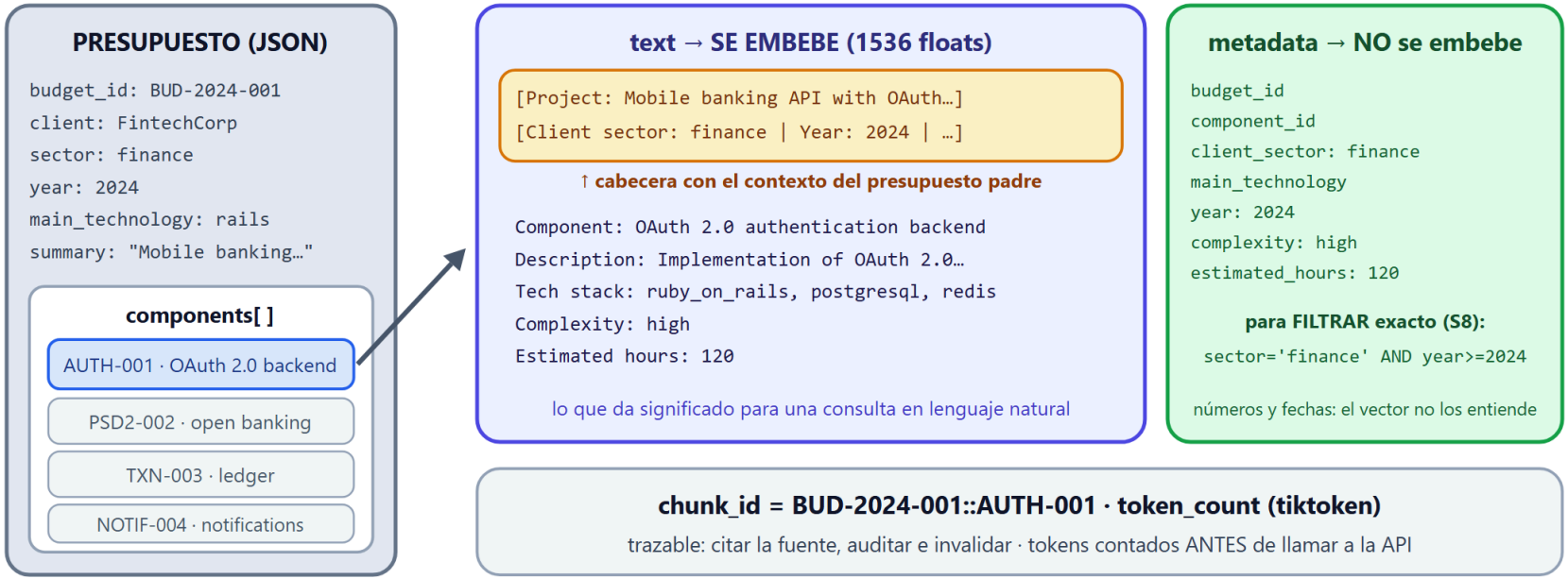
Buscar = comparar el vector de la consulta con el de cada chunk y quedarse con los más cercanos.

Si un chunk está mal cortado, su vector lo representa mal… **y ningún modelo de embeddings lo arregla después.**

# El chunk de nuestro proyecto

app/generation/rag/chunking/structural.py · la estrategia de referencia

## Anatomía de un chunk: qué se embebe y qué viaja al lado




Las otras siete estrategias reutilizan este mismo render de texto: **todas trocean la misma representación del corpus**, así la comparación es justa.

# Por qué el chunking domina

Un presupuesto entero en un vector es un promedio difuso; un trozo demasiado pequeño pierde el contexto

### Vectara · NAACL 2025

25 configuraciones × 48 modelos

Cambiar el chunking mueve la calidad **tanto como cambiar de modelo**.

### Chroma · 2025

recall por estrategia

LLM semantic **0,919** · cluster semantic **0,913** · recursive bien ajustado (400 tokens) **0,88–0,89**.

### Vecta · febrero 2026

ranking sobre su benchmark

Recursive 512 tokens, **primero con un 69 %**. Semantic, **cuarto con un 54 %**.

### Más sofisticado no significa mejor

La única respuesta honesta a "¿qué chunking es el mejor?" es **medirlo sobre tus datos** antes de comprometer arquitectura. Es lo que hacemos hoy.

L1DR

# Las cuatro familias de chunking

Ocho estrategias agrupadas por cómo deciden dónde partir el texto

## 1 Mecánicas

Cortan por reglas · ignoran el contenido

Bloque 4 — 30 min

- FixedSize — baseline degenerado
- Recursive — default razonable
- SentenceWindow — oración indexada · ventana devuelta

Coste: $0

## 2 Estructurales

Explotan el formato del documento

Bloque 4 (parcial) + Bloque 6

- JSONStructural — un componente = un chunk
- Hierarchical — parent-child · dos niveles

Coste: $0

## 3 Semánticas

Cortan donde cambia el significado

Bloque 5 — primera mitad

- Semantic — breakpoint por caída de similitud
- Propositional — LLM descompone en proposiciones

Coste real · medir ROI

## 4 Contextuales

Enriquecen el chunk antes de embedear

Bloque 5 + Bloque 6

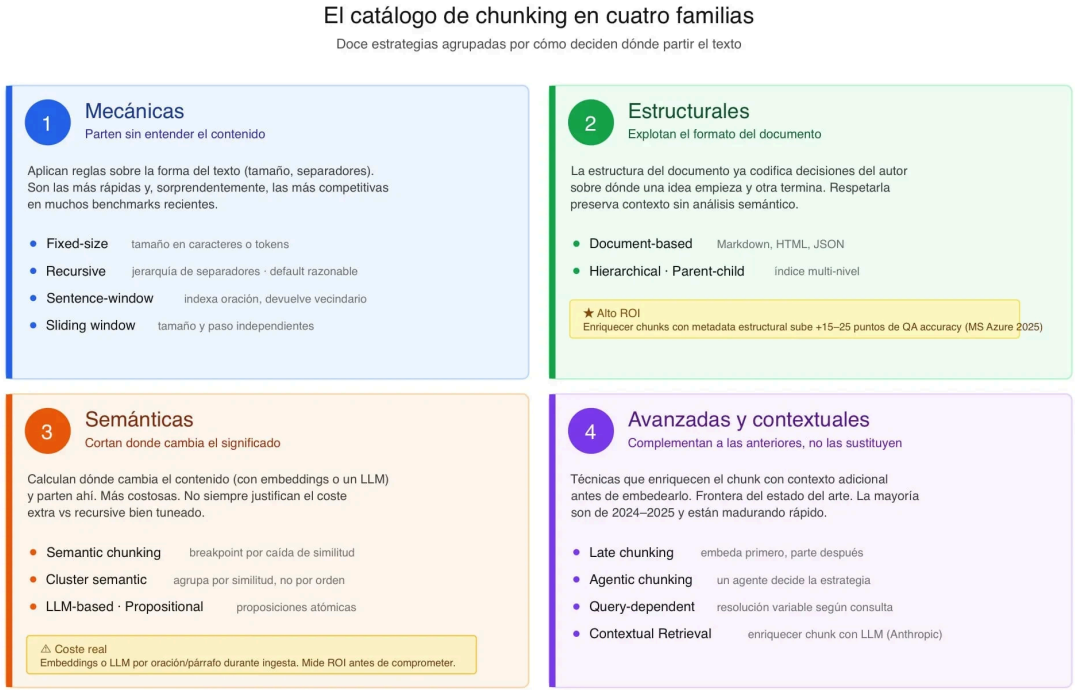
- ContextualRetrieval — Claude + prompt caching
- LateChunking — demo conceptual

Coste premium

Todas implementan la misma interfaz Chunker · intercambiables en el comparador

# Doce estrategias, cuatro familias

Hoy ejecutamos ocho sobre nuestro corpus, todas tras la misma interfaz Chunker




**chunking/base.py · la interfaz común**

```
class Chunker(ABC):
    @abstractmethod
    def chunk(self, budgets) -> list[Chunk]:
        ...
    # coste de ingesta, por estrategia
    last_extra_api_calls: int = 0
    last_extra_cost_usd: float = 0.0
```

## Siete criterios para elegir

- Empieza con **recursive**, 400–512 tokens, 10–20 % de overlap.
- Si hay estructura, **úsala**: la palanca de mayor ROI.
- Chunks sin sentido aislados → Contextual Retrieval.
- Semantic y LLM-based, solo con datos que lo justifiquen.
- Parent-child, sobre un pipeline plano que ya funcione.
- Late, agentic, query-dependent: vigilar, no producir.
- Cada tipo de documento pide su estrategia.

L1DR

# Las columnas del Chunking Lab

Qué significa cada estadística y cuándo es buena o mala señal

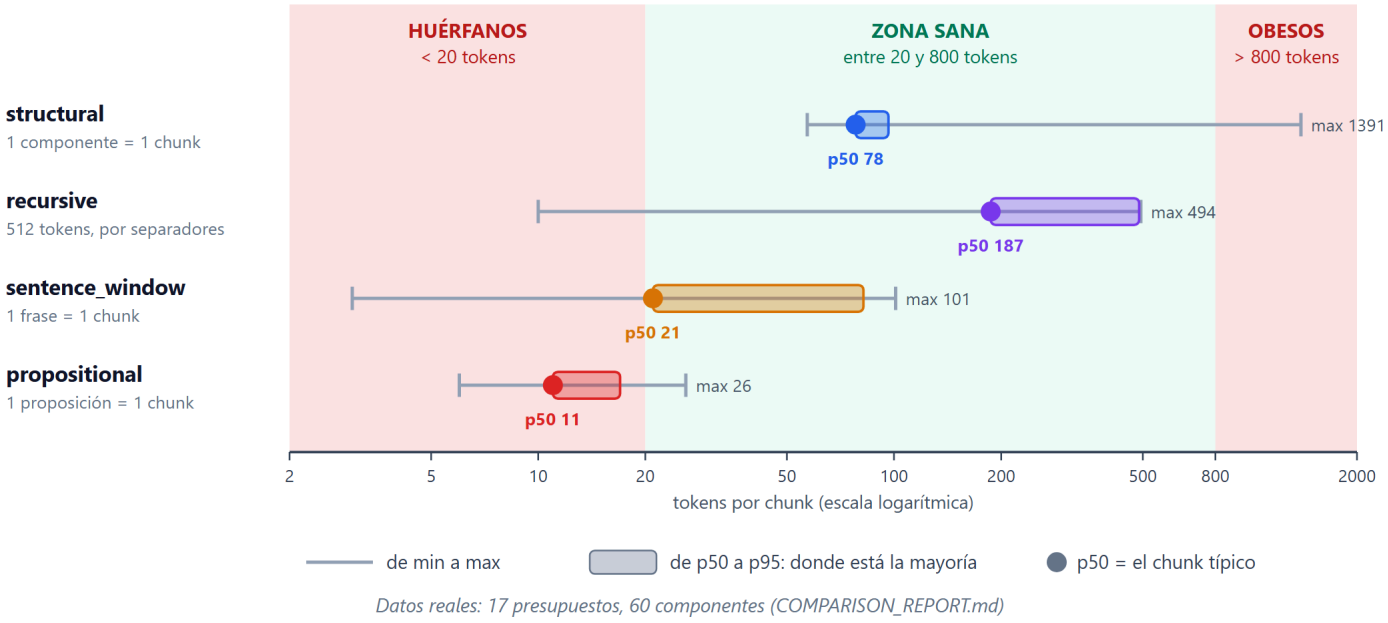
| Columna | Qué es | ¿Bueno o malo? |
|---|---|---|
| **Chunks** | cuántos trozos (y vectores) produce la estrategia | ni bueno ni malo: más chunks = índice más grande y más coste de embeber |
| **min · max** | el chunk más pequeño y el más grande, en tokens | un max muy alto avisa de un obeso; un min muy bajo, de huérfanos |
| **p50** | la **mediana**: la mitad de los chunks mide menos que esto. El chunk típico | conviene que caiga en la zona sana, lejos de los dos extremos |
| **p95** | el 95 % de los chunks mide menos que esto (p90: lo mismo con el 90 %) | si está muy lejos de p50, los tamaños son irregulares |
| **Huérfanos** | chunks de **menos de 20 tokens** | **malo**: sin contexto, dan coincidencias con score alto que no sirven |
| **Obesos** | chunks de **más de 800 tokens** | **malo**: mezclan temas, su vector es un promedio difuso y llenan el contexto del LLM |
| **Coste $ · Segundos** | lo que cuesta trocear el corpus (la ingesta) | solo las estrategias que llaman a un LLM o a embeddings cuestan algo |

No hay un número mágico. Lo que buscamos: **cero huérfanos, cero obesos** y un p50 que tenga sentido para el tipo de documento.

# Leer las estadísticas sobre nuestro corpus

El mismo corpus, cuatro formas de partirlo

## Tamaño de los chunks en tokens: dónde cae cada estrategia




*Datos reales: 17 presupuestos, 60 componentes (COMPARISON_REPORT.md)*

**structural**

p50 78, p95 96,5: muy homogéneo. Un obeso: `MONO-001`, 1391 tokens.

**sentence_window**

p50 21: la mitad roza el límite. **34 huérfanos** de 78.

**propositional**

p50 11: **765 huérfanos** de 778. Mucho score, poco contexto.

Recursive es el más irregular (de 10 a 494 tokens): corta a 512 tokens por separadores sin mirar dónde empieza un componente.

L1DR

# Fixed-size vs Recursive

Mismo presupuesto serializado · dos formas radicalmente distintas de partirlo

## FixedSize · CHUNK_SIZE_TOKENS = 512 · OVERLAP = 80

BUD-2024-016::MONO-001::p0 · 512 tokens

```
[Project: Migration of legacy banking platform...] Component:
Core banking monolith decomposi tion into 18 microservices
Description: Strangler-fig migration of a 12-year-old Java mono
lith handling 2.4M daily transactions. The migration plan spa
ns 18 months and inclu
```

BUD-2024-016::MONO-001::p1 · 512 tokens

```
des the following dom ains: account management, transactional
core, identity services, fraud detection, audit, regulatory r
eporting, KYC compliance, AML monitoring, customer profiling,
notification dispatch, scheduled batches, and external integ
rations with PSD2-com
```

BUD-2024-016::MONO-001::p2 · 367 tokens

```
pliant providers. Each domain will be implemented as an in
dependent service with its own database and event bus...
[...]
```

63 chunks · p50: 79 tok · max: 512 tok · obesos: 0

"decomposi tion" · "dom ains" · palabras rotas a mitad

## Recursive · separadores [\n\n, \n, ". ", " ", ""]

BUD-2024-016::r0 · 488 tokens

```
[Project: Migration of legacy banking platform to cloud-native
microservices architecture]

Component: Core banking monolith decomposition into
18 microservices.
Description: Strangler-fig migration of a 12-year-old Java...
```

BUD-2024-016::r1 · 494 tokens

```
...handling 2.4M daily transactions.

The migration plan spans 18 months and includes the
following domains: account management, transactional core,
identity services, fraud detection, audit, regulatory
reporting, KYC compliance...
```

BUD-2024-016::r2 · 187 tokens

```
...AML monitoring, notification dispatch, scheduled batches,
and external integrations with PSD2-compliant providers.
```

23 chunks · p50: 187 tok · max: 494 tok · obesos: 0

Oraciones intactas · respeta saltos naturales del lenguaje

**El default razonable: Recursive con 400–512 tokens y 10–20% de overlap**

Solo se complejiza con evidencia medida de que no es suficiente · MVP con un día de plazo: este es el chunker

L1DR

# Sentence-window vs Structural

Misma consulta · resultados que parecen iguales pero esconden la trampa del score alto

CONSULTA: **"OAuth authentication for fintech mobile app"**

## SentenceWindow · indexa oraciones sueltas

#1 [0.4530] BUD-2024-001::PSD2-002::s0
"Implements PSD2 strong customer authentication..."
relevante

#2 [0.4410] BUD-2024-017::MVP-001::s0
"Email and password login."
⚠ del MVP de e-commerce — score alto, contexto perdido

#3 [0.4110] BUD-2024-006::VENDOR-001::s0
"Third-party vendor onboarding with SSO..."
parcialmente relevante

## JSONStructural · un componente = un chunk

#1 [0.5684] BUD-2024-001::AUTH-001
[Project: Mobile banking API...] OAuth 2.0 backend...
✓ el componente OAuth correcto con contexto

#2 [0.5310] BUD-2024-001::NOTIF-004
[Project: Mobile banking API...] Notification svc...
mismo proyecto · relacionado

#3 [0.5220] BUD-2024-001::PSD2-002
[Project: Mobile banking API...] PSD2 SCA module...
✓ relacionado, mismo dominio

### Lo que el sentence-window oculta

Indexa oraciones diminutas y descontextualizadas · "Email and password login." parece auth, no lo es
La oración recuperada NO es lo que pasarías al LLM-generador · eso vive en **`metadata.window_text`**
Score alto ≠ chunk relevante. La unidad indexada debe ser distinta de la devuelta cuando la indexada pierde sentido aislada.

# Las cuatro mecánicas, medidas

compare_chunkers.py --strategies fixed-size,recursive,sentence-window,structural --queries all --output-table

**Demo · ejecución real (6 consultas)**

| strategy | n_chunks | p50_tokens | top1_cosine_avg |
|---|---|---|---|
| fixed_size | 63 | 79.0 | 0.4760 |
| recursive | 23 | 187.0 | 0.3991 |
| sentence_window | 78 | 21.0 | 0.5366 |
| structural | 60 | 78.0 | 0.4734 |

## Los parámetros

- **fixed-size** / **recursive**: 512 tokens, overlap 80.
- **recursive**: corta primero por párrafo, luego por línea, frase y palabra.
- **sentence-window**: indexa la frase, guarda ±2 frases en `metadata.window_text`.

## Cómo leerla

`top1_cosine_avg` = la confianza media del mejor candidato en las 6 consultas.

Sentence-window saca el número más alto… y para "OAuth authentication for fintech mobile app" cuela **"Email and password login."** en su top-3. **Score alto ≠ chunk relevante.**

## Sin contar coste, ¿cuál preferís?

- **Structural**: misma confianza que fixed-size, pero respeta la unidad de negocio.
- **Recursive**: el default razonable cuando no hay estructura.
- **Sentence-window**: útil en prosa sin estructura, con su ventana para generar.

# Semantic chunking: el coste real

Embebe frases consecutivas y corta donde cae la similitud

**Dónde vive**

```
# chunking/strategies/semantic.py
from langchain_experimental.text_splitter import SemanticChunker
BREAKPOINT_THRESHOLD_TYPE = "percentile"
BREAKPOINT_THRESHOLD_AMOUNT = 95   # solo los saltos bruscos
```

**Demo · --show-stats --show-cost**

| strategy | n_chunks | p50 | max | cost_usd | seconds |
|---|---|---|---|---|---|
| semantic | 34 | 112.5 | 1278 | 0.000094 | 5.31 |
| structural | 60 | 78.0 | 1391 | 0.000000 | 0.00 |

## Qué hace

Junta lo que va junto temáticamente: **34 chunks** frente a 60. El percentil 95 es agresivo: pocos cortes y chunks grandes.

## Qué cuesta

Una pasada de embeddings por **todas las frases** del corpus, antes de servir una sola consulta. Con 17 presupuestos es ridículo; con un millón, esos 5 segundos son horas.

## ¿Lo aprobáis en una review?

Depende de si la mejora lo justifica **en tu corpus**. Sobre presupuestos estructurados, structural ya separa bien; semantic brilla en prosa larga sin estructura.

# Propositional chunking: precisión a cambio de contexto

Un LLM descompone cada componente en proposiciones atómicas y autocontenidas

**Dónde vive**

```
# chunking/strategies/propositional.py
PROPOSITIONAL_CHUNKER_MODEL = "gpt-4o-mini"
class _Propositions(BaseModel):      # salida tipada
    propositions: list[str]          # vía Instructor
# si una llamada falla → 1 chunk = el componente entero
```

## Consulta "real-time inventory synchronization"

**0,823** `BUD-2024-007::INV-003::prop5`

el score más alto de toda la sesión: la proposición clavada

## El precio, sobre el corpus completo

| | |
|---|---|
| Chunks | **778** (13× structural) |
| Huérfanos (< 20 tokens) | **765** |
| Coste de ingesta | 0,00755 $ |
| Tiempo de ingesta | **173 s** |

En la demo se limita a 3 presupuestos: 149 chunks en 28 s.

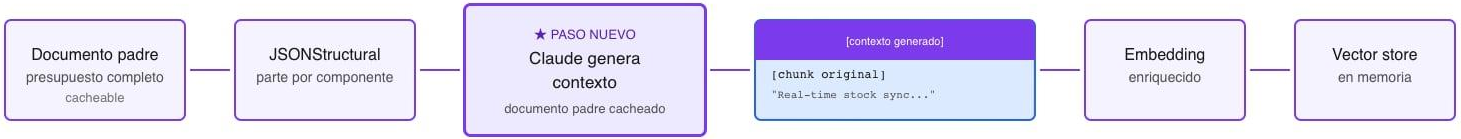
## ¿Para qué corpus sí y para cuál no?

- **Sí**: corpus densos en hechos donde manda la precisión factual (legal, médico).
- **No**: corpus ya estructurados. La explosión de fragmentos sin contexto no compensa.

L1DR

# Contextual Retrieval

Enriquece el chunk con un párrafo generado por Claude · prompt caching del documento padre




## RESULTADO SOBRE NUESTRO CORPUS · consulta: "OAuth authentication for fintech mobile app"

| | | |
|---|---|---|
| JSONStructural (baseline)<br>Top-1: BUD-2024-001::AUTH-001 | 0.5684 | |
| ContextualRetrieval<br>Mismo chunk + contexto generado por Claude | 0.6070 | +0.04 |

## COSTE REAL · 60 chunks · 17 presupuestos

| | | |
|---|---|---|
| JSONStructural | | $0.00 · 0 segundos |
| ContextualRetrieval | | $0.140 · 202 segundos (3.4 min) |

+0.04 de cosine por $0.14 · 3.4 minutos de ingesta · ¿lo aprobáis en una review?

**Anthropic reporta hasta –67% de fallos de retrieval combinado con BM25 contextual + reranking**

El header estático del JSONStructural ya es Contextual Retrieval barato · la versión LLM solo paga la pena en corpus donde compensa medir

# Contextual Retrieval en el código

Un párrafo generado por Claude sitúa cada chunk en su presupuesto

**Dónde vive**

```
# chunking/strategies/contextual_retrieval.py
CONTEXTUAL_CHUNKER_MODEL = "claude-sonnet-4-5"
MAX_CONTEXT_TOKENS = 200

system=[{"type": "text",
         "text": f"<document>\n{parent_doc}\n</document>",
         "cache_control": {"type": "ephemeral"}}]
# el contexto generado → metadata.generated_context
```

## Prompt caching

Los chunks se procesan **agrupados por presupuesto**: el primero paga el documento (escritura en caché, 3,75 $/M), los demás lo leen de caché (**0,30 $/M**, unas 10× más barato).

## Nuestro structural ya lleva una cabecera…

`[Project: …] [Client sector: … | Year: … | Main tech: …]`

Es una versión **estática y gratis** de la misma idea. La pregunta honesta: ¿cuánto añade la versión con LLM?

## Sobre nuestro corpus

AUTH-001: **0,568** → **0,607** (+0,04)
60 chunks · **0,14 $** · **203 s** de ingesta

No es obvio que compense aquí. Esa discusión coste/valor es la lección.

# Una consulta transversal: todas convergen

"GDPR compliance and audit logging" · top-1 de cada estrategia

| Estrategia | Top-1 | Coseno |
|---|---|---|
| structural | `BUD-2024-009::CONSENT-004` | **0,418** |
| fixed_size | `BUD-2024-009::CONSENT-004::p0` | **0,418** |
| sentence_window | `BUD-2024-009::CONSENT-004::s0` | **0,617** |
| semantic | `BUD-2024-009::sem1` | **0,358** |
| propositional | `BUD-2024-009::CONSENT-004::prop5` | **0,673** |
| contextual_retrieval | `BUD-2024-009::CONSENT-004` | **0,465** |
| hierarchical | `BUD-2024-009::CONSENT-004` | **0,418** |

## La señal está

Todas aterrizan en **CONSENT-004**, el módulo de consentimiento y auditoría de un proyecto de sanidad. Lo que cambia es la confianza y el tamaño de lo que devuelven.

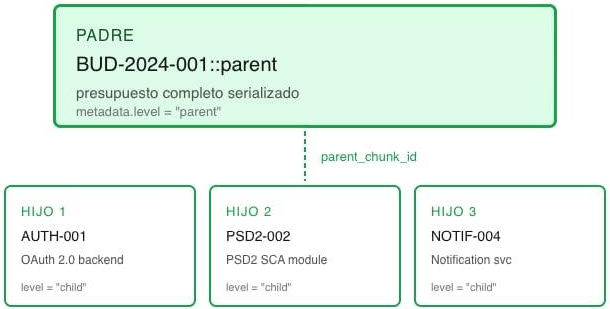
## ¿Cuál gana?

Pregunta trampa: ganan distinto según qué optimices.

- **Score**: propositional.
- **Contexto**: contextual.
- **Coste**: structural, 0 $.
- **Legibilidad**: structural.

# Hierarchical · parent-child

Dos niveles indexados a la vez · el hijo da el recall, el padre da el contexto




CONSULTA: **"GDPR compliance and audit logging"**

#1 HIJO [0.4180] `BUD-2024-009::CONSENT-004`

`[Project: Healthcare data platform...] Consent and audit log...`

↓ con --resolve-parents activado, se devuelve también el padre BUD-2024-009::parent

#2 PADRE [0.3070] `BUD-2024-012::parent`

`presupuesto completo serializado`

⚠ un padre entero entra en el top-3 sin necesidad de resolver

### Hijos para recall

Granularidad fina · matchea
consultas específicas con score alto.

El embedding del componente es preciso y discriminante.

### Padres para contexto

Cuando el hijo matchea, se
devuelve también el padre completo.

Útil para queries abstractas: sube el padre al top-3 por sí solo.

### ¿Cuándo activar?

✓ Manuales largos
✓ Documentos legales
✗ FAQs cortos
✗ Tickets breves · presupuestos: borderline

Stats: 77 chunks = 17 padres + 60 hijos · obesos: 2 (max 1472) · coste: $0

# Hierarchical en la terminal

--strategies hierarchical --queries "GDPR compliance and audit logging" --show-top-k 3 --resolve-parents

**Padre e hijos en el mismo índice**

```
# chunking/strategies/hierarchical.py
Chunk(chunk_id=parent_id, ...,
      metadata={"level": "parent"})
Chunk(...,
      metadata={"level": "child",
                "parent_chunk_id": parent_id})
```

**Demo · ejecución real**

```
hierarchical: 77 chunks (17 padres + 60 hijos)
  2 obesos (max 1472) · 0 $
1. [0.4180] BUD-2024-009::CONSENT-004  (hijo)
2. [0.3070] BUD-2024-012::parent   ← padre entero
3. [0.3010] BUD-2024-012::EXPORT-003
```

## Lo interesante

Para una consulta abstracta, **un presupuesto completo** entra en el top-3. A veces el padre entero es mejor respuesta que un componente suelto.

El retriever de la Sesión 8 decidirá qué nivel devolver.

## ¿Cuándo activarlo?

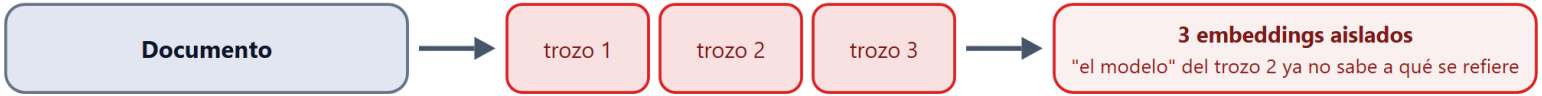
- **Útil**: manuales largos, documentos legales.
- **Overkill**: FAQs, tickets cortos.
- **Presupuestos**: en el límite; probablemente útil para consultas abstractas.

# La que no ejecutamos hoy

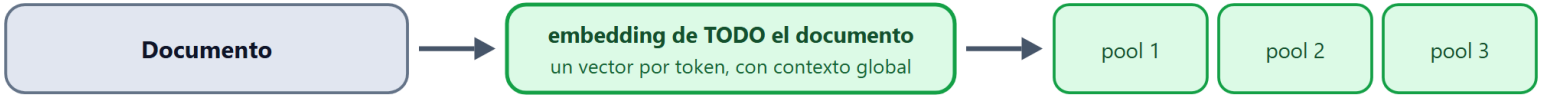
Nuestro modelo de embeddings no da vectores por token

## Late chunking: cambiar el orden de las operaciones

**Tradicional: primero se parte, luego se embebe cada trozo**




**Late chunking: primero se embebe el documento entero, luego se parte**




cada chunk = media de SUS tokens, que ya vieron todo el documento

**Qué necesita**

un modelo con ventana larga (8K–32K tokens)
y que exponga embeddings por token (p. ej. jina-embeddings-v3)

**En nuestro proyecto**

text-embedding-3-small no expone embeddings por token:
hoy es concepto, no demo · técnica emergente (Jina, 2024)

*Útil cuando el significado de cada parte depende mucho del documento completo*

**¿Cuándo compensa?** Documentos largos, modelo con contexto grande y re-embeber barato. En nuestro caso, todavía no.

# El cuadro de mando: toda la sesión en una tabla

compare_chunkers.py · 17 presupuestos · huérfano < 20 tokens · obeso > 800 tokens

| Estrategia | Chunks | p50 | Huérf. | Obesos | $ | Ingesta |
|---|---|---|---|---|---|---|
| structural | 60 | 78 | 0 | 1 | 0 | 0,00 s |
| fixed_size | 63 | 79 | 0 | 0 | 0 | 0,00 s |
| recursive | 23 | 187 | 1 | 0 | 0 | 0,01 s |
| sentence_window | 78 | 21 | 34 | 0 | 0 | 0,01 s |
| semantic | 34 | 112,5 | 0 | 1 | 0,000094 | 5,31 s |
| propositional | 778 | 11 | 765 | 0 | 0,00755 | 173 s |
| contextual_retrieval | 60 | 132,5 | 0 | 1 | 0,140 | 203 s |
| hierarchical | 77 | 81 | 0 | 2 | 0 | 0,01 s |

## Tres lecturas

- **Coste**: de 0 $ a 0,14 $.
- **Granularidad**: de 23 a 778 chunks.
- **Degeneración**: de 0 a 765 huérfanos.

## La pregunta del día

Si fuera vuestro proyecto en producción: **¿con qué estrategia salís hoy?**

Y más importante: **¿qué mediríais antes de cambiarla?**
recall@k formal, coste por consulta, latencia… (Sesión 11)

La métrica de hoy es informal: estadísticos + coseno del top-3 + inspección. No es recall@k, y lo decimos abiertamente.

L1DR

# Lo que cierras hoy · lo que abre la S8

## SALIMOS HOY CON

- Ocho estrategias ejecutadas sobre tu corpus
- Métricas comparativas: stats + cosine + cualitativa
- Una estrategia elegida con justificación
- Plan de qué medirías antes de cambiarla
- 60 chunks vectorizados · en memoria

EN MEMORIA · HTTP

## LA SESIÓN 8 AÑADE

- PostgreSQL + pgvector
- Índices HNSW / IVFFlat para escala
- Filtrado por metadata + similitud combinada
- Búsqueda semántica real a baja latencia
- Persistencia que sobrevive al restart

PERSISTENCIA · ÍNDICES

## EL ARCO COMPLETO DEL MÓDULO

SESIÓN 06 Corpus limpio → SESIÓN 07 ✓ Chunking + vectores → SESIÓN 08 ← siguiente pgvector + retrieval → SESIÓN 10 Retrieval avanzado → SESIÓN 11 Generación y calidad

El código de hoy se queda intacto · la S8 añade una capa, no sustituye nada

# ✋Tiempo para el feedback con ROTI

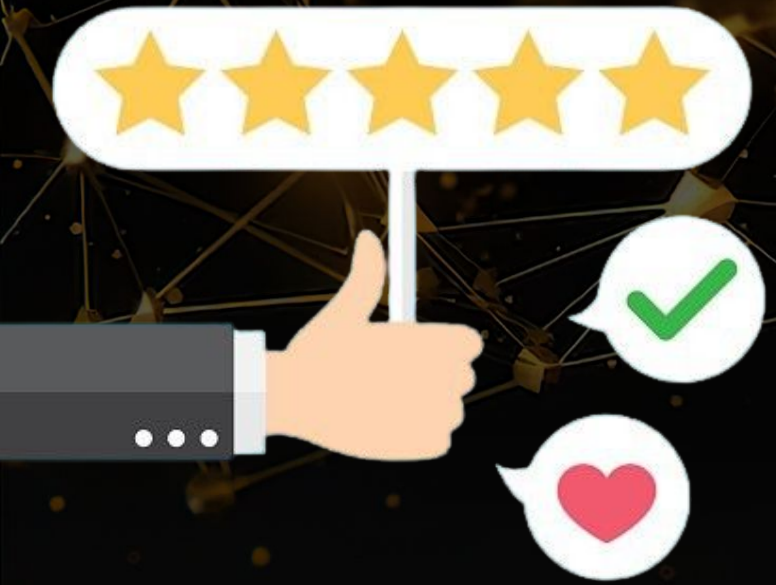
## ¿Cuánto valor te ha agregado esta sesión?



| Absolutamente nada | Poco | Suficiente | Bastante | Más de lo que esperaba |
|---|---|---|---|---|

LIDR

# ROTI



# ¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR